A I P O C K E T A G E N C Y

aipocketagency.com

[apa · kit 05]

Wire the Brain to Your Stack

Seven MCP walkthroughs — Drive, Gmail, Slack, Notion, Linear, GitHub, Supabase — with auth setup, working queries, real gotchas, and the brain-sync patterns that turn a static brain into a system that pulls from your actual work.

KIT	PRICE	FORMAT
05 / 05	$15 USD	12 pages

Chase Whited · 2026-05-12

TABLE OF CONTENTS

SECTION 1.

Why the brain multiplies with MCPs

A brain repo without MCPs is a private wiki. A brain repo with seven MCPs wired is a system that pulls from your actual work and writes back to it. The walk from one to the other is what this kit is.

The first month with a brain repo, the value is obvious — file-based memory, agent rules in CLAUDE.md, decision log entries that survive auto-compact. The agent stops forgetting. The agent stops contradicting itself. The agent reads CLAUDE.md and behaves better than it did before. That's the wedge.

The second month is when the brain plateaus. The memory entries you wrote in week one are still there. They're still accurate. But they aren't updating. The brain is a snapshot of what you knew in week one, and the gap between the snapshot and reality grows every day. You start writing memory entries by hand to keep it current. Then you stop. The brain becomes a museum.

MCPs are the fix. An MCP is the wiring that lets an agent read from and write to a specific tool — Google Drive, Gmail, Slack, Notion, Linear, GitHub, Supabase. When the agent can pull a Drive folder's contents at session start, the brain stops being a static snapshot. When the agent can write a brain Decision Log entry every time a Slack thread crosses the "this is a decision" threshold, the brain stops being a wiki you maintain by hand. The brain becomes the system that pulls from your actual work and writes back to it.

The seven MCPs in this kit are the ones I have wired across every project I run. They cover where decisions happen (Slack, Linear), where work product lives (Drive, GitHub, Notion), where customer communication runs (Gmail), and where production data lives (Supabase). Other MCPs exist; some matter for specific projects. These seven are the foundation.

> **WHAT THIS KIT ASSUMES**
>
> You already have a brain repo set up. If you don't, start with the Dev-Team Document Set kit or the CLAUDE.md Template Library kit. This kit assumes you have a `whited-brain`-style repo with `CLAUDE.md`, `AGENTS.md`, and a memory directory in place — and you're ready to wire it to the tools where work actually happens.

SECTION 2.

Before you start — the universal setup

Three things are true for every MCP. Get them right once and the per-MCP walkthroughs go fast.

1. MCP config lives in a known file

Every modern agent reads MCP configuration from a JSON file at a stable path. For Claude Code on macOS, that's `~/.claude/mcp.json`. For other agents, the path varies — Cursor reads `~/.cursor/mcp.json`, Codex reads its own equivalent. The kit ships a one-page "where MCP config lives per agent" reference card.

The file is a JSON object with one key per server name. Each entry includes the command to spawn the server, the args, and (where needed) the environment variables. A real entry looks like this:

```
{
  "mcpServers": {
    "drive": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-google-drive"],
      "env": {
        "GOOGLE_OAUTH_CREDENTIALS": "/Users/you/.config/mcp/drive-oauth.json"
      }
    }
  }
}
```

Keep the file small. The temptation is to wire every MCP under the sun on day one. The trap is that each MCP loads its tool catalog into the agent's context at session start, and ten poorly-scoped MCPs can swallow the first thousand tokens of every conversation. Wire what you'll use. Skip what you won't.

2. Secrets live in 1Password, never in the config file

Every MCP needs credentials. The wrong move is to paste tokens into `mcp.json` and check the file into git. The right move is to resolve secrets at use time — the kit ships the `getsecret` shell helper that reads `op://Personal/<Service>/<field>` references via the 1Password CLI. The MCP config references the secret by path, not by value. The token never lands on disk.

```
# In your shell config (~/.zshrc):
export GITHUB_TOKEN="$(getsecret op://Personal/GitHub/personal-access-token)"
```

For MCPs that take credentials via OAuth callback (Drive, Gmail), the OAuth flow writes a refresh token to disk by design. Move that file to a known config dir under `~/.config/mcp/` and chmod it to `600`. Never commit it.

3. Test each MCP with a known-good query before wiring it to anything

Every MCP walkthrough in this kit ends with three example queries. Run them. They're calibrated to fail loud if something is wrong — wrong auth scope, missing permission grant, expired token. Get one green query before you wire any sync automation. A sync automation built on top of an MCP that doesn't actually work is the worst kind of silent failure.

SECTION 3.

Drive MCP

Auth, three queries, three gotchas, one sync pattern. Wires your Google Drive to the brain.

Auth. Create a Google Cloud project in the console at `console.cloud.google.com`. Enable the Drive API. Create an OAuth 2.0 Client ID for "Desktop app." Download the JSON. Move it to `~/.config/mcp/drive-oauth.json` (chmod 600). Run the MCP server once locally; it will open a browser, you'll grant access, and the refresh token will be saved alongside the OAuth credentials JSON.

Scope selection matters. For reading existing files, `drive.readonly` is enough. If you want the agent to be able to update a doc — say, append a decision-log row to a Drive-hosted Decision Log — you need `drive.file` (which limits the agent to files it created or that you explicitly granted via picker) or `drive.metadata` for listing. Avoid `drive` (full access) unless you have a specific reason.

Three example queries (paste these into your agent after the MCP is wired):

```
1. "Find the latest decision log in shared drive 'Whited Consulting' —
    return the file ID and a 200-character snippet of the most recent
    entry."

2. "List the 10 most recently modified files in folder 'AOS / Phase 18
    Planning' — return name, last-modified timestamp, and the user who
    modified it."

3. "Pull the full contents of the doc titled 'Patrick Discovery Call —
    2026-04-12' and return it as plain markdown."
```

Three gotchas:

1. **Shared-drive vs My-drive scope.** The default Drive API queries hit "My Drive" only. Files in a shared drive don't appear unless you set `corpora=allDrives` and `includeItemsFromAllDrives=true` in the request. Most MCP servers expose a flag for this; check yours.
2. **OAuth token expiry.** Refresh tokens last for six months of inactivity. If your MCP sits unused for that long, the next call will silently fail with an opaque error. The fix is to re-run the OAuth flow. Set a calendar reminder.
3. **Rate limits.** Drive API rate-limits aggressively on large-folder listing. If you're scanning a folder with 10,000 files, batch with `pageSize=100` and back off on 429s.

Sync pattern — "Drive changelog → brain Decision Log entry." The pattern: an agent watches a designated "decisions" folder in Drive; when a new doc lands there with a `[DECISION]` prefix in the title, it generates a brain Decision Log entry referencing the Drive doc ID and a short summary. The kit ships the markdown template and the prompt that does the conversion.

SECTION 4.

Gmail MCP

Auth, three queries, three gotchas, one automation pattern. Wires your inbox to the brain.

Auth. Same Google Cloud project as Drive (or create a new one). Enable the Gmail API. Use the same OAuth Client ID — Gmail and Drive can share. Move the credentials to `~/.config/mcp/gmail-oauth.json`. First run opens a browser; refresh token saves.

Scope: `gmail.readonly` for reading. `gmail.modify` if you want the agent to apply labels (this is the most useful write scope — the agent can label a thread "decision-pending" and the label becomes the trigger for a sync pattern). Avoid `gmail.send` unless you have a specific reason — the kit's email-drafting pattern uses Drive + a manual review step rather than direct send.

Three example queries:

```
1. "Summarize all threads in label 'Client / Patrick' in the last 7
    days. For each thread, return: subject, last-message timestamp,
    last-message sender, and a 50-word summary."

2. "Find all emails from 'patrick@freshpagehome.com' in the last 30
    days and return them as a list of (date, subject, snippet) tuples."

3. "Pull the most recent thread containing 'storm season prep' —
    return all messages in chronological order, including any
    attachments' filenames."
```

Three gotchas:

1. **Scope creep risk.** Gmail OAuth scopes can't be tightened later; you have to revoke and re-grant. Pick the minimum scope on day one. If you later need to bump to `gmail.modify`, do it deliberately and re-grant on the OAuth screen.
2. **Attachment handling.** Most Gmail MCPs return attachment metadata but not the bytes by default. Fetching the bytes is a second call. Plan for it — large attachments can blow the agent's context budget if you fetch them naively.
3. **Message-vs-thread ID confusion.** A thread has one ID; each message in the thread has its own ID. Queries that ask "show me the last message" need the thread ID, then a `messages.get` against the last message ID. Most MCP layers expose both — use threads for context, messages for specific replies.

Automation pattern — "Gmail label → brain task entry." A Gmail filter applies the label `brain-task` to threads that match a keyword pattern. A scheduled agent run checks for

labeled threads, creates a corresponding `TASKS.md` entry in the brain, and removes the label. The brain becomes the durable task list; Gmail becomes the inbox trigger.

SECTION 5.

Slack MCP

Auth, three queries, three gotchas, one pattern. Wires your team's decisions to the brain.

Auth. Slack MCP setup runs through the Slack App console at `api.slack.com/apps`. Create an app for your workspace. Under "OAuth & Permissions," grant the bot scopes you need — for read-only summaries, `channels:history`, `groups:history`, `mpim:history`, `im:history`, `users:read`. Install the app to the workspace. Copy the bot token (starts with `xoxb-`) into 1Password. Reference it from the MCP config via `getsecret`.

User token vs bot token is the decision that catches people. The bot token (`xoxb-`) is correct for most cases — the bot acts as itself, joins channels you invite it to, and can read history in those channels. The user token (`xoxp-`) lets the bot act as you, which is occasionally useful (e.g., to read all your DMs) but carries serious permission baggage. Use the bot token unless you have a specific reason.

Three example queries:

```
1. "Summarize channel #engineering this week. For each day, return a
    one-paragraph summary of the decisions discussed and links to the
    most-reacted message of the day."

2. "Find all messages in #leadership last month that contain the word
    'decision' or '[DECISION]' — return as (timestamp, author,
    permalink, text) tuples."

3. "Pull message context for thread URL https://slack.com/archives/...
    — return the parent message, all replies in chronological order,
    and the participants list."
```

Three gotchas:

1. **Bot-vs-user token gotchas.** The bot token can only read channels the bot is a member of. Inviting the bot to a private channel requires the channel admin to add it. Don't be surprised when the bot reports "no messages" for a channel it isn't in.
2. **Private channel access.** Even with the right scopes, the bot needs to be explicitly added to each private channel. The kit ships a setup script that lists every channel the bot is missing from, so you can do the invites in one pass.
3. **Rate limits on history endpoints.** `conversations.history` is tier 3 rate-limited (50 requests per minute). Bulk-scanning a year of channel history will hit the limit fast. Batch by week or month.

Pattern — "Slack decision message → brain Decision Log entry." Tag a Slack message with the 📋 reaction (or any agreed emoji). A scheduled agent run finds all messages with that reaction since the last sweep, generates Decision Log entries linking back to each Slack permalink, and clears the reaction once the entry is written. The Slack reaction is the trigger; the brain Decision Log is the durable record.

SECTION 6.

Notion MCP

Auth, three queries, three gotchas, one pattern. Wires your Notion workspace to the brain.

Auth. Notion uses integration tokens, not OAuth. Create an integration at `notion.so/my-integrations`. Pick "Internal" unless you're shipping a public app. Copy the secret (starts with `secret_`). Store it in 1Password. Reference it from `mcp.json` via `getsecret`.

The critical second step: the integration must be invited to each page or database you want to access. Notion's permission model is per-page, not workspace-wide. Open the page, click "Add connections" in the top-right menu, pick your integration. Skip this and every query returns "object not found."

Three example queries:

```
1. "Find all rows in database 'Client CRM' where Status = 'Active'.
    Return as a table with columns: Company, Primary Contact, Last
    Touch, MRR."

2. "Pull the full content of page 'Q2 2026 Strategy' — return as
    markdown with all sub-pages flattened inline."

3. "List the 20 most recent edits across the entire workspace — return
    page title, edit timestamp, and the user who edited."
```

Three gotchas:

1. **Integration must be invited per page or database.** Already noted — but the failure mode is silent. The MCP returns "no results" instead of a clear "you don't have access." If a query returns empty unexpectedly, check the invite first.
2. **Rate limits.** Notion's API rate-limits at 3 requests per second. A bulk import that paginates aggressively will hit the limit. Back off with exponential delays.
3. **Block-vs-page model.** A Notion page is a tree of blocks. Reading "a page" actually means fetching every block in the tree. Long pages with embedded databases or sub-pages can take multiple paginated calls. Plan for it — a single `get_page` is often three to five real API calls.

Pattern — "brain → Notion sync." Push brain changelog entries to a designated Notion database. The pattern is one-directional (brain → Notion), not bidirectional. The brain is the source of truth; Notion is the readable view for teammates who don't open the brain repo. The

kit ships the prompt that does the conversion plus a `last-synced-at` watermark in the brain so you don’t re-push old entries.

SECTION 7.

Linear MCP

Auth, three queries, three gotchas, one pattern. Wires your issue tracker to the brain.

Auth. Linear supports both an API key (simplest) and OAuth (for multi-user apps). For solo and small-team use, the API key is the right call. Create one at `linear.app/settings/api`. Scope it to "Personal" — not "Workspace" — unless you specifically need cross-workspace access. Store in 1Password.

Three example queries:

```
1. "List all open issues assigned to me in project 'AOS Phase 18' —
    return as (identifier, title, priority, days-since-last-update)
    tuples."

2. "Find all issues with label 'tech-debt' in project 'wc-admin' —
    return total count plus a histogram by priority."

3. "Create a new issue in project 'APA' with title '<title>', body
    '<body>', priority Medium, and label 'kit-build'. Return the new
    issue identifier."
```

Three gotchas:

1. **Project-vs-team scoping.** Linear's API has both "team" (the human team in your org) and "project" (a body of work within a team). Most queries you'll write target a project; team-level queries return everything in the team. Be explicit about which scope you want — the wrong scope returns either too much or nothing.
2. **Label conventions vary by team.** If you have a team that uses `tech-debt` and another that uses `Tech Debt` (capitalized), your label query will miss one of them. Standardize labels across teams before relying on label-based sync.
3. **Issue-state transitions.** Linear has workflow states (`Backlog` , `In Progress` , etc.) and the agent can move issues across states via the API. Be careful — agents that "advance" issues based on heuristic checks have a way of moving the wrong issue out of `Done` and into `In Progress` . Require explicit confirmation for state transitions.

Pattern — "Linear ticket → brain task entry; closure → Linear close." Bidirectional sync between Linear issues and brain `TASKS.md` . When a Linear issue is created with a specific label (e.g., `brain-sync`), the agent creates a corresponding TASKS.md entry. When the brain entry is marked closed, the agent closes the Linear issue. The watermark file is a `linear-brain-sync.json` log so duplicates don't happen.

SECTION 8.

GitHub MCP

Auth, three queries, three gotchas, one pattern. Wires your repos to the brain.

Auth. Two flavors — Personal Access Token (PAT) for single-user use, GitHub App OAuth for multi-user or org-level use. For the solo operator use case this kit is calibrated for, the PAT is the right call. Create a fine-grained PAT at `github.com/settings/tokens?type=beta`. Scope it to specific repositories — not "All repositories" — to limit blast radius if the token leaks. Store in 1Password.

Three example queries:

```
1. "List the 20 most recent commits on repo 'whited-brain' — return
    SHA, author, date, and the first line of the commit message."

2. "Find all PRs touching the file 'CLAUDE.md' in repo 'athlete-hub-
    pro' — return PR number, title, status, and merge date if merged."

3. "Summarize commit SHA a7f3e21 in repo 'aipocketagency-website' —
    return the files changed, line counts per file, and a one-
    paragraph summary suitable for a changelog entry."
```

Three gotchas:

1. **PAT scope creep.** Fine-grained PATs are scoped per-repo on creation; you can't add a repo later without recreating the token. Plan the scope on day one. If you operate across many repos, accept that you'll have multiple PATs.
2. **GitHub App installation per repo.** If you're using a GitHub App instead of a PAT, the app must be installed on each repo. Org-level installation is possible but requires admin. For most solo use, the PAT path is faster.
3. **Rate limits on graph queries.** GitHub's GraphQL API rate-limits by "node count" rather than raw request count. A single deep query can burn the entire hourly budget. The fix is to batch and project — ask for the fields you need, not `*`.

Pattern — "commits + PR descriptions → brain Change Log entries." A scheduled agent run scans the last 7 days of commits on each tracked repo, groups them by PR, and generates Change Log entries summarizing each merged PR. The agent reads the PR description and title — operator-written context — rather than inferring from the diff. The brain Change Log becomes the human-readable history of shipped work across every repo.

SECTION 9.

Supabase MCP

Auth, three queries, three gotchas, one pattern. Wires your database to the brain.

Auth. Supabase MCP uses an access token from `supabase.com/dashboard/account/tokens`. The access token grants project-level admin — keep it close. For most queries, you'll want to use the anon or service-role keys per-project rather than the master access token. The MCP server typically accepts both; check which mode your server uses before scoping permissions.

Three example queries:

```
1. "List all tables in project 'AOS Production' — return table name,
    row count estimate, and the RLS policy count per table."

2. "Run SELECT id, name, created_at FROM profiles WHERE
    created_at > now() - interval '7 days' LIMIT 50 on project 'AOS
    Production' — return as a markdown table."

3. "Check migration status on project 'wc-admin' — return the most
    recently applied migration name plus any migrations in the local
    repo that have not been applied."
```

Three gotchas:

1. **RLS bypass risk.** The service role key bypasses all Row Level Security. An agent with a service-role connection can read every row in every table — including PII it shouldn't see in a debugging session. Use the anon key with an explicit JWT for most queries; reserve service role for migration tasks and similar.
2. **Schema-drift detection.** The MCP can list tables and columns, but it can't tell you the live schema matches the migrations folder. Periodic schema drift between staging and prod is a real problem; the kit ships a `schema-drift.sh` script that compares.
3. **Migration apply order.** Supabase applies migrations alphabetically by filename. Two migrations with the same timestamp prefix collide. Name migrations with timestamps to the second (`20260512143022_*.sql`), not just date.

Pattern — "Supabase schema → brain Feature Inventory sync." The agent walks the live Supabase schema once a week and updates the brain's Feature Inventory with current table counts, RLS policy counts, and migration tip. The Feature Inventory becomes the readable picture of what the database actually holds, sourced from the database itself rather than from somebody's memory.

SECTION 10.

What comes next

The seven walkthroughs above are the foundation. The patterns for stitching them together — cross-MCP orchestration — are the community deep-dive.

The kit gives you the wiring. The wiring multiplies the brain — a Drive folder becomes a Decision Log feed, a Slack reaction becomes a Decision Log entry, a Linear ticket becomes a brain task, a GitHub PR becomes a Change Log line. Each MCP earns its place in the agent's session context, and the brain stops being a museum.

What the kit doesn't cover, by design: writing your own MCP server (when off-the-shelf doesn't exist), cross-MCP orchestration patterns (stitching three MCPs into one workflow), and the brain dashboard wiring that surfaces all of this as live signal. Those live in the AI Pocket Agency community at `aipocketagency.com` — $47/month, locked for life for the founding 50. Members get free updates when the MCP landscape shifts (and it shifts every quarter).

If you've already shipped the kit's seven walkthroughs and you want to extend, the community is the next stop. If you haven't, run them in the order this kit lists — Drive and Gmail first (the data-pull MCPs), then Slack and Linear (the decision-capture MCPs), then Notion and GitHub (the work-product MCPs), then Supabase last (the production-state MCP). Each one builds confidence for the next.

Wire one. Watch the brain pull from your actual work. Wire the next.

— Chase